



Target - crackme_solid_04.exe

Eugenio D. Deluca (KarasuR00T) – Date: 13/08/2024

Hello! Here I am again, continuing to learn this whole reversing thing. Well, here's the deal: I have a crackme named crackme_solid_04.exe.

At first glance, I found that the file seems to be written in C++ and compiled in 64-bit. Now, when I execute it, this appears:

```
C:\> Administrador: C:\Windows\system32\cmd.exe
Microsoft Windows [Versión 6.1.7601]
Copyright (c) 2009 Microsoft Corporation. Reservados todos los derechos.

C:\Users\SQL-PC>cd c:\revvv

c:\revvv>crackme_solid_04.exe
Bienvenidos al curso de Reversing 2024
Para empezar, decime tu nombre: pepe
Hola pepe vamos a aprender reversing !
falta algo para seguir avanzando...

c:\revvv>
```

Well, according to my understanding, that means something is "missing," so I decided to open it with IDA.

```

; int __fastcall main(int argc, const char **argv, const char **envp)
main proc near

    dwCreationDisposition= dword ptr -58h
    dwFlagsAndAttributes= dword ptr -50h
    hTemplateFile= qword ptr -48h
    var_38= dword ptr -38h
    FileName= byte ptr -30h
    var_20= qword ptr -20h

; __unwind { // __GSHandlerCheck
push    rsi
push    rdi
sub     rsp, 68h
mov     rax, cs:__security_cookie
xor     rax, rsp
mov     [rsp+78h+var_20], rax
lea     rcx, aBienvenidosAlC ; "Bienvenidos al curso de Reversing 2024"...
call    sub_13F9812B0
mov     ecx, 3E8h ;
call    cs:Sleep
call    sub_13F981070
lea     rax, [rsp+78h+FileName]
lea     rcx, aTestTxt ;
mov     rdi, rax
mov     rsi, rcx

```

```

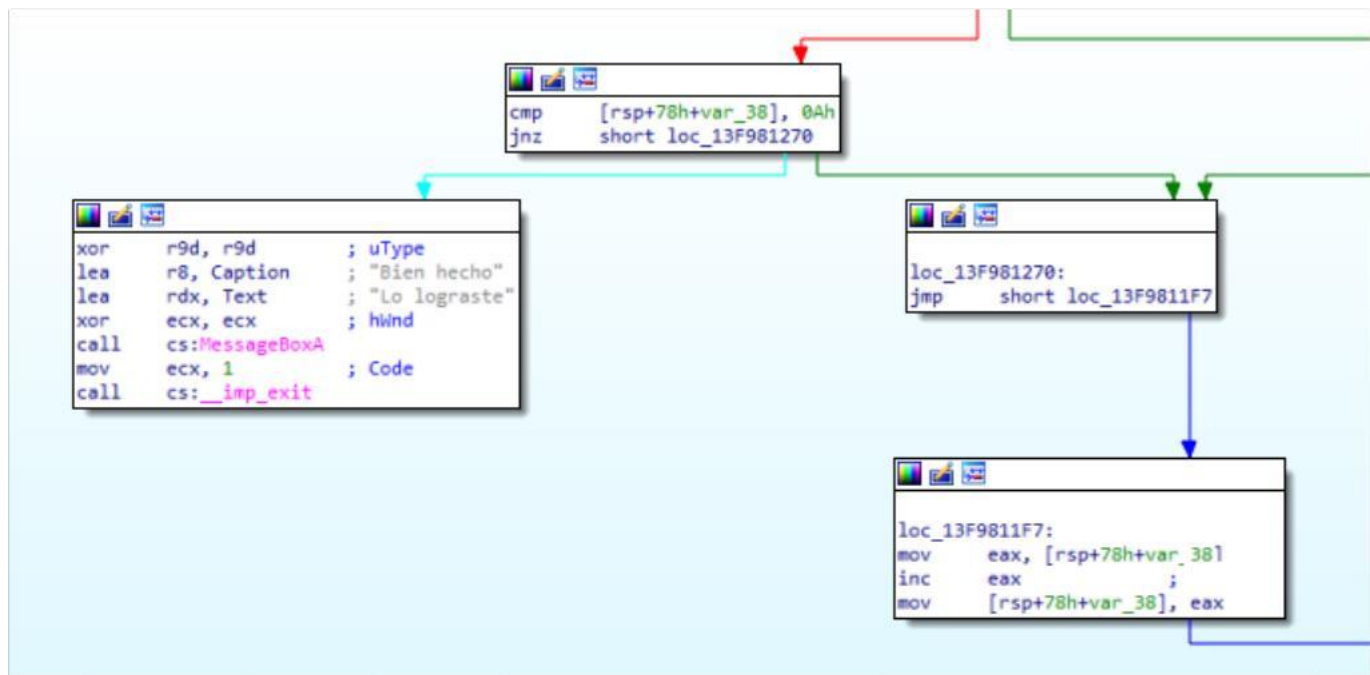
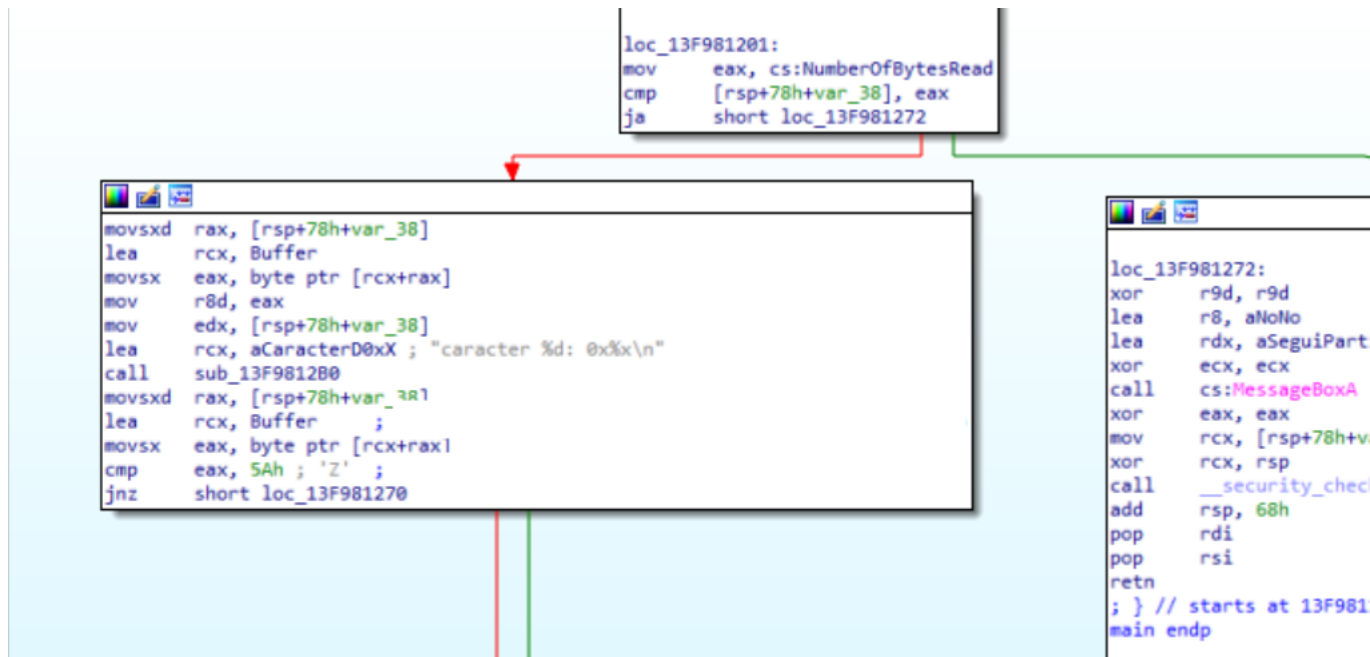
mov     ecx, 9 ;
rep movsb
mov     [rsp+78h+hTemplateFile], 0 ; hTemplateFile
mov     [rsp+78h+dwFlagsAndAttributes], 80h ; dwFlagsAndAttributes
mov     [rsp+78h+dwCreationDisposition], 3 ; dwCreationDisposition
xor     r9d, r9d ; lpSecurityAttributes
xor     r8d, r8d ; dwShareMode
mov     edx, 0C0000000h ; dwDesiredAccess
lea     rcx, [rsp+78h+FileName] ; lpFileName
call    cs:CreateFileA ;
mov     cs:qword_13F985040, rax ;
cmp     dword ptr cs:qword_13F985040, 0FFFFFFFFh ;
jnz     short loc_13F9811E1

```

```

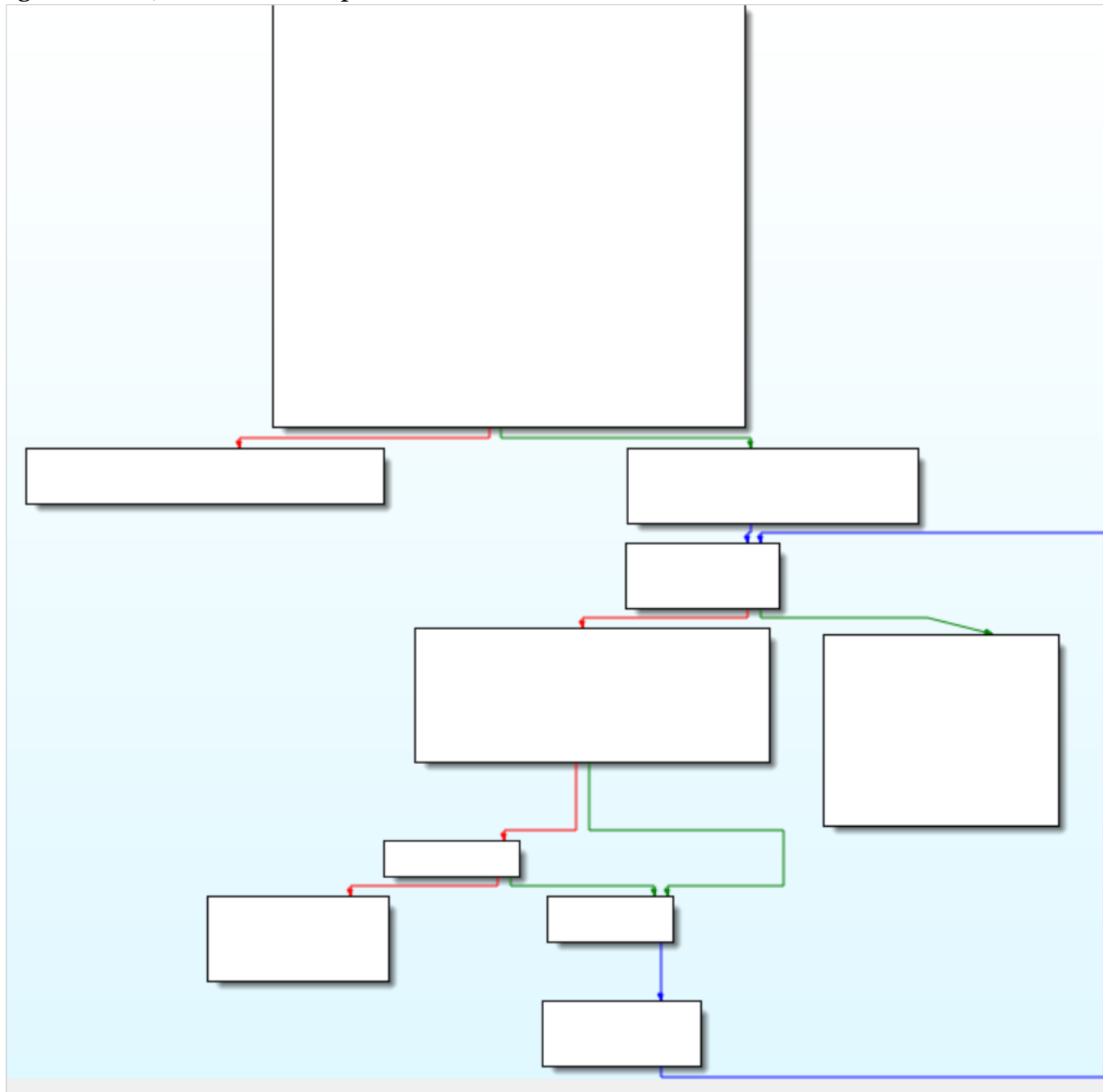
lea     rcx, aFaltaAlgoParaS ; "falta algo para seguir avanzando...\n"
call    sub_13F9812B0
mov     ecx, 1 ;
call    cs:__imp_exit

```



Pido disculpas pero no tenía otra forma más que sacar varias capturas.

I apologize, but I had no choice but to take several screenshots. To give you a more "global" idea, here is the complete flowchart:



Alright, here are some details I noticed at first glance:

```
; __unwind { // __GSHandlerUnk  
push    rsi  
push    rdi  
sub     rsp, 68h  
mov     rax, cs:__security_cookie  
xor     rax, rsp  
mov     [rsp+78h+var_20], rax  
lea     rcx, aBienvenidosAlC ; "Bienvenidos al curso de Reversing 2024"...  
call    sub_13F8A12B0  
mov     ecx, 3E8h           ; imprime bienvenida y espera 1 seg  
call    cs:Sleep
```

The call labeled sub_13F8A12B0 seems to be a printf

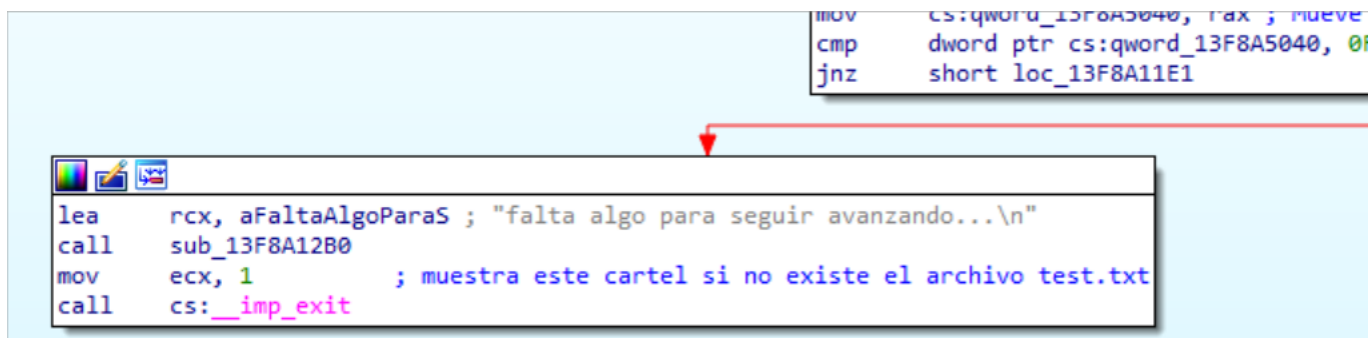
```
var_10= qword ptr -10h
; __unwind { // __GSHandlerCheck
push    rdi
sub     rsp, 40h
mov     rax, cs:__security_cookie
xor     rax, rsp
mov     [rsp+48h+var_10], rax
lea     rax, [rsp+48h+Buffer]
mov     rdi, rax
xor     eax, eax
mov     ecx, 14h
rep stosb
lea     rcx, aParaEmpezarDec ; "Para empezar, decime tu nombre: "
call    sub_13F8A12B0
mov     edx, 14h             ; Size
lea     rcx, [rsp+48h+Buffer] ; Buffer
call    cs:gets_s
lea     rdx, [rsp+48h+Buffer]
lea     rcx, aHolaSVamosAApr ; "Hola %s vamos a aprender reversing !\n"
call    sub_13F8A12B0
mov     rcx, [rsp+48h+var_10]
```

This is the image of that call -> when setting a breakpoint (BP) and executing it, it matches the moment it prints that welcome banner.

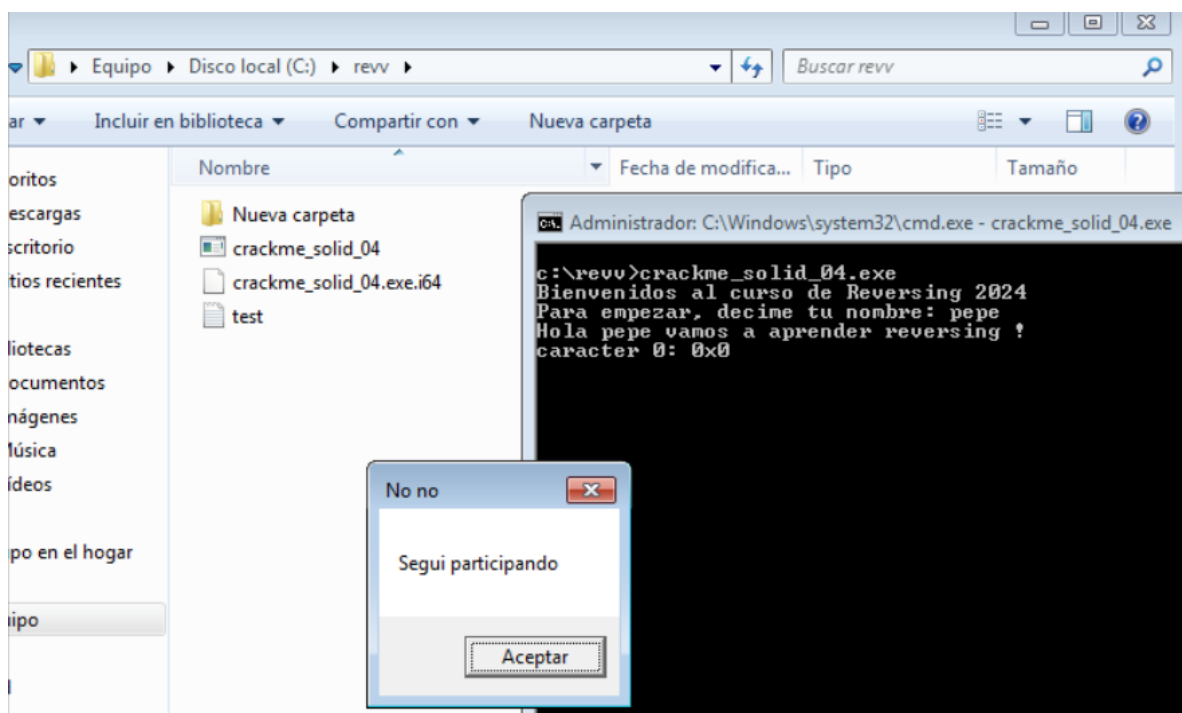
Continuing with the tracing, I notice it indicates the following:

```
call    sub_1400012B0
mov     ecx, 3E8h             ; dwMilliseconds
call    cs:Sleep
call    sub_140001070
lea     rax, [rsp+78h+FileName]
lea     rcx, aTestTxt         ; "test.txt"
mov     rdi, rax
mov     rsi, rcx
```

Another clue is that IDA shows this jump (preceded by a comparison):

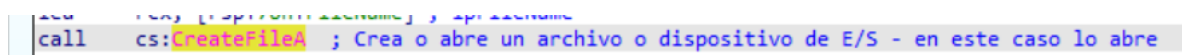


I think it's looking for a file named **"test.txt"** there. So, I think the best option would be to create that file in the same directory where the crackme is located and see what happens:



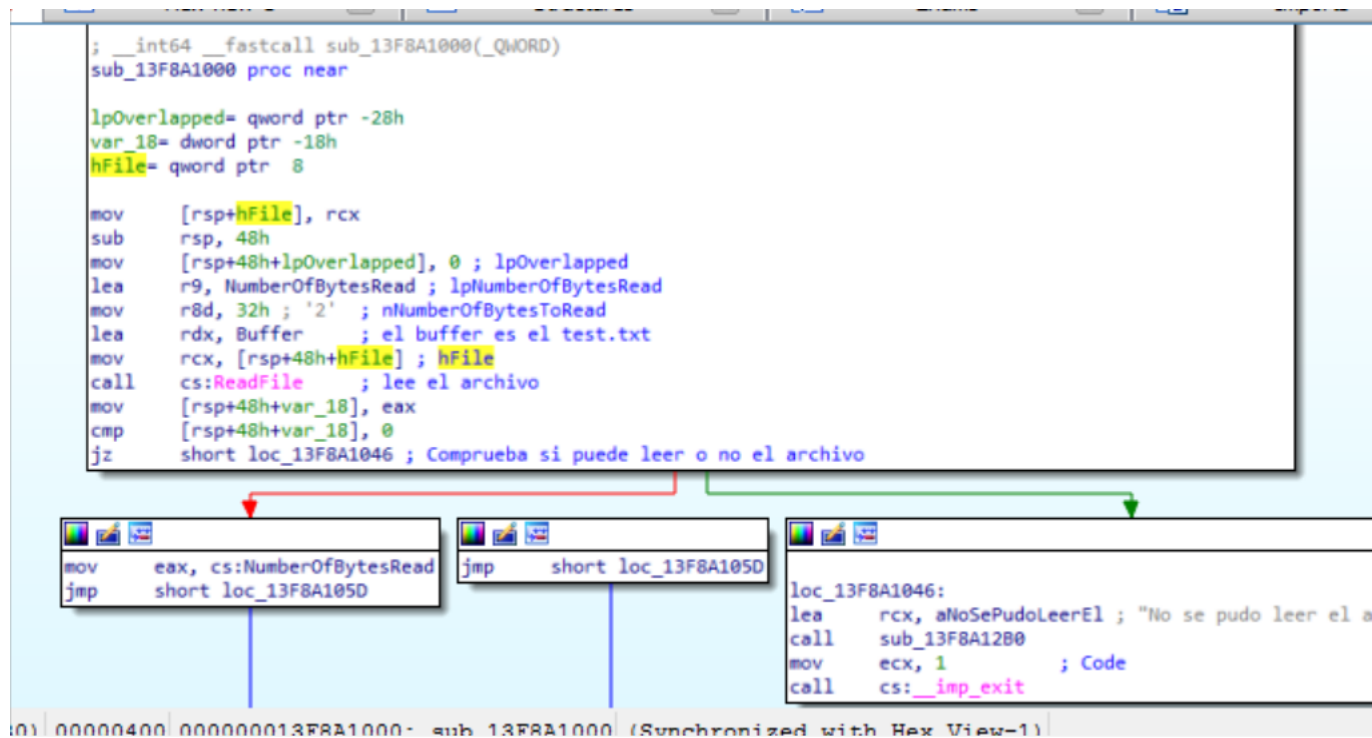
Okay, we managed to bypass the previous banner, but we still haven't hit the **GOOD BOY** message. So, so far we have: a crackme that kindly welcomes you to a course (?) and asks you for a name, prints a greeting, and then checks whether the file test.txt exists or not -> If it exists, it returns "character 0: 0x0" and a nice little banner.

A few details I skipped:

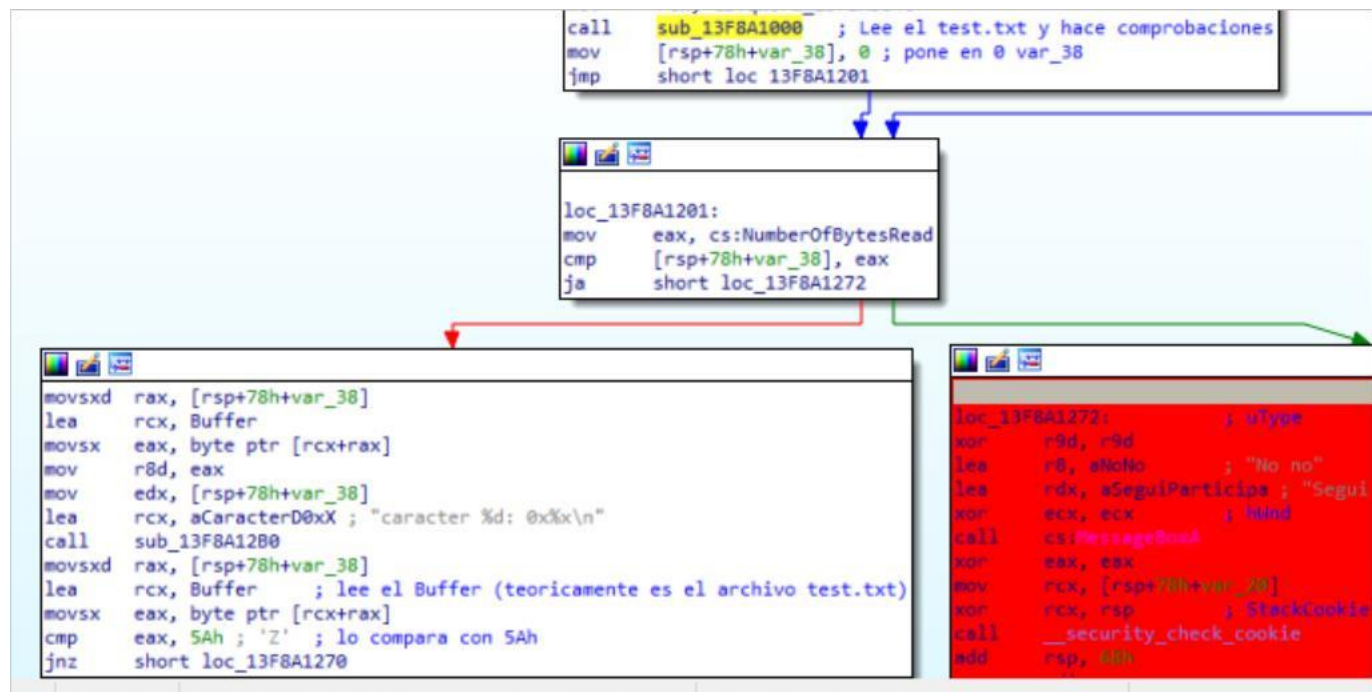


According to MSDN, the CreateFileA function creates or opens a file or I/O device. From what we've been seeing, in this instance, it doesn't create anything; rather, it opens said file.

Another thing I noticed is that the call sub_13F8A100 performs certain verifications:



Alright, so I keep looking at IDA—first of all, I'll mark the **BAD BOY** banner ->



Looking at it this way, there's a detail that catches my attention:

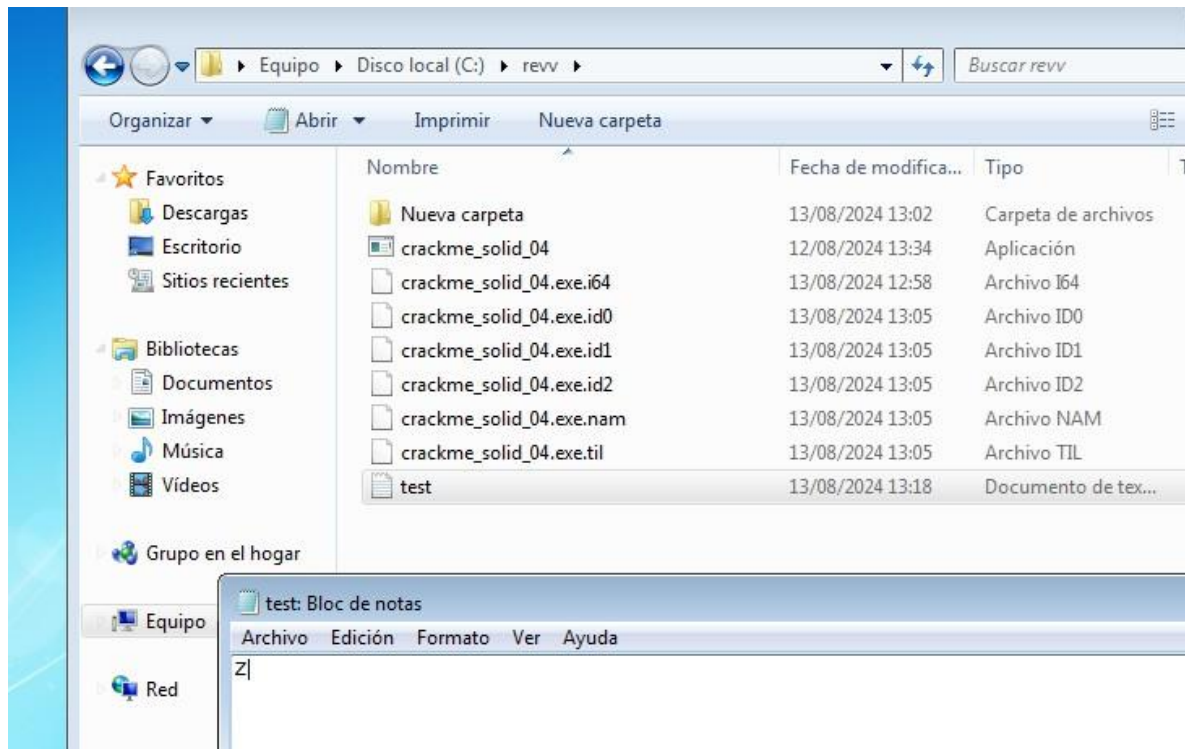
```

cmp     eax, 5Ah ; 'Z' ; lo compara con 5Ah
jnz     short loc_13F8A1270

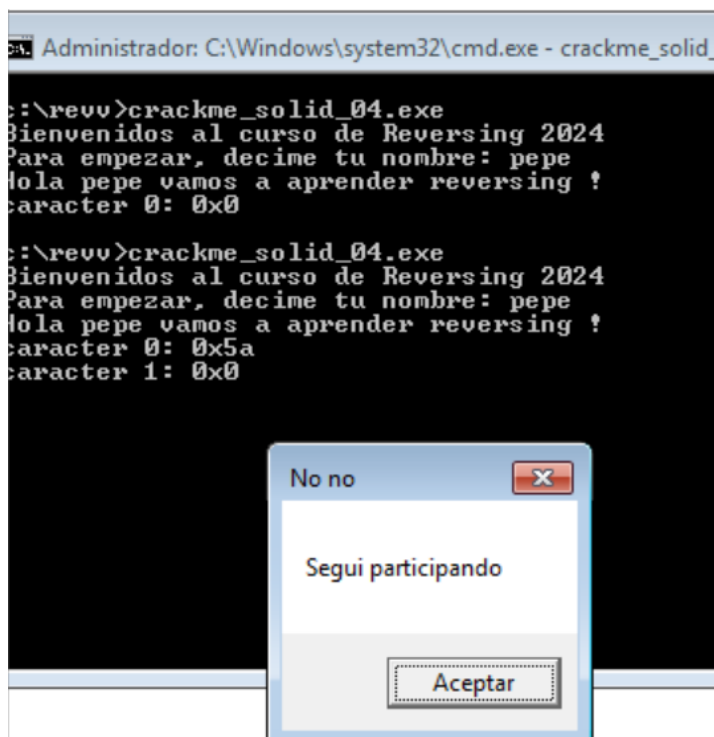
```

That right there! What is it?! In other words, it compares the content of EAX with 5A (which is Z in hex).

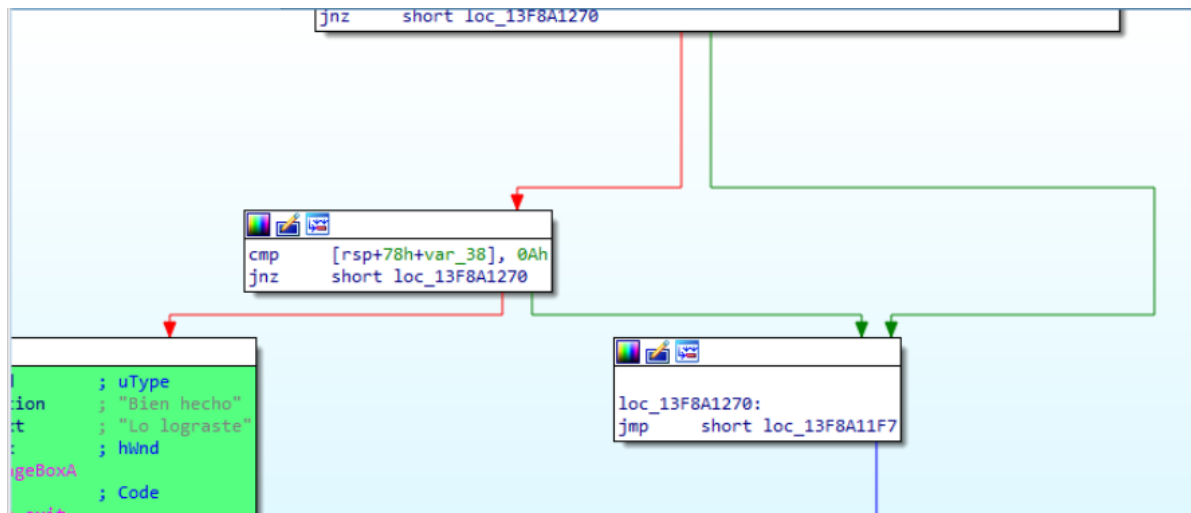
Well, here we have a clue. I'm going to put a Z (keep in mind it's a capital Z, not lowercase) inside the txt file and see what it does.



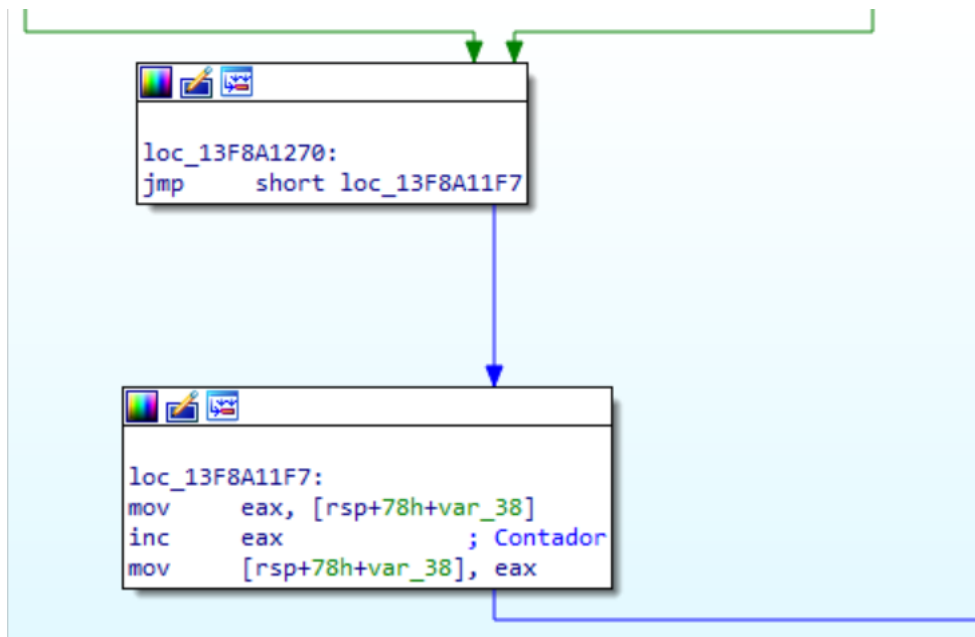
Whoa!!



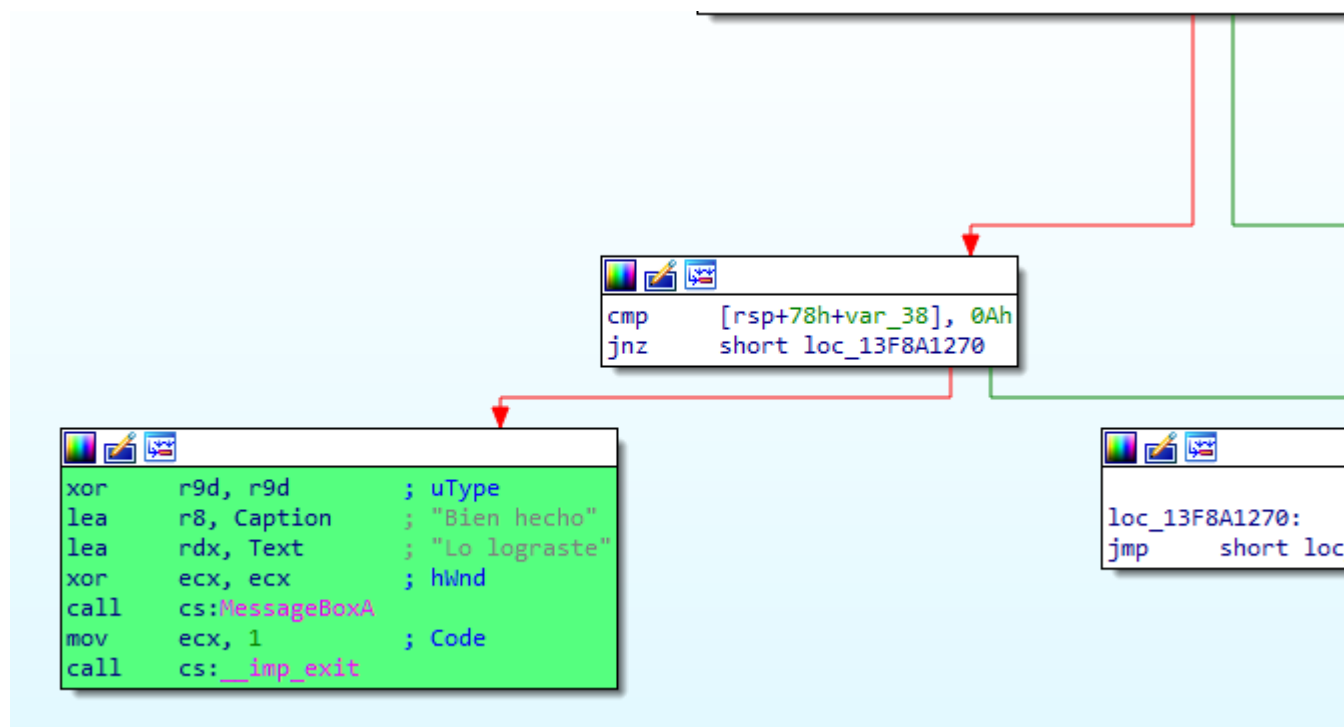
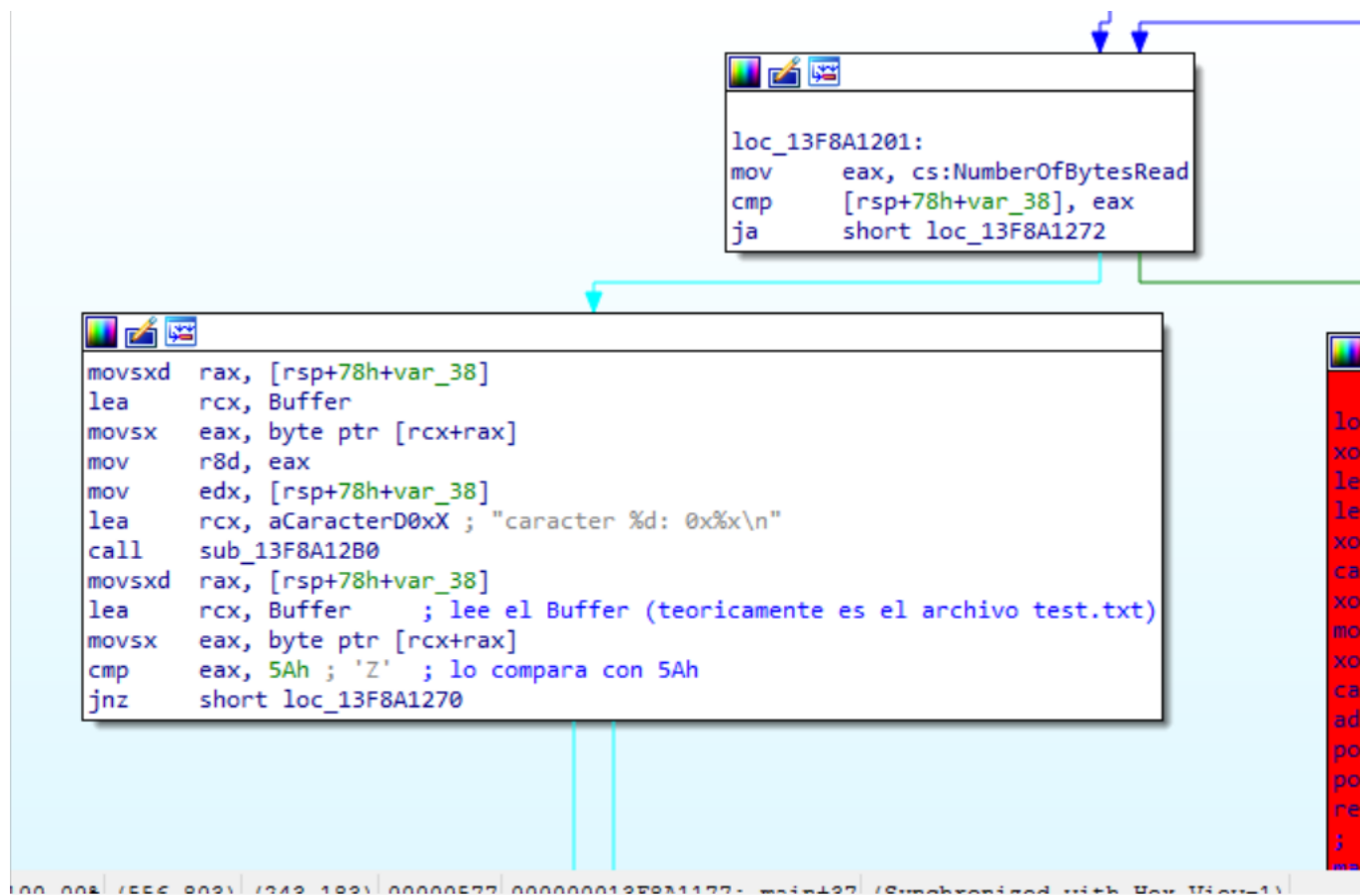
What happened there? While we still get the "Keep trying" (*Segui participando*) banner, it indicates character 0x5a (meaning I was somewhat right)—but it also says "character 1: 0x0", and I'm sure I only put one Z in the file. Okay, back to IDA.



Here I see the `jnz` jump -> and then it compares it again against 0A (which is 10) and has two paths -> it either exits through the Good Boy banner (which we haven't reached yet) or goes through the other `jmp` □



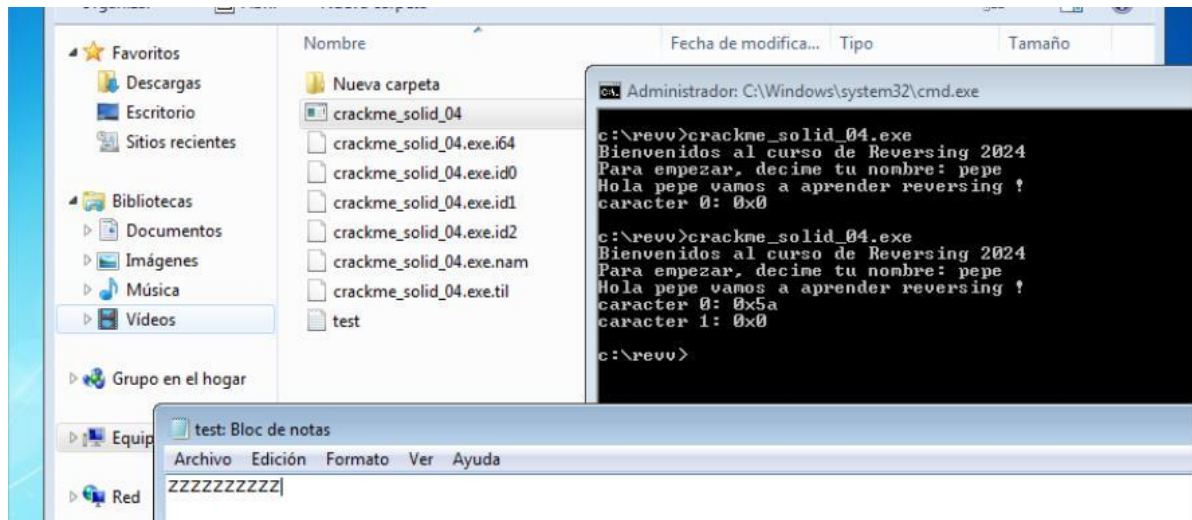
Okay, I see that those jumps end up at a counter (meaning `var_38` is highly likely a counter)— and that jump then loops back here:



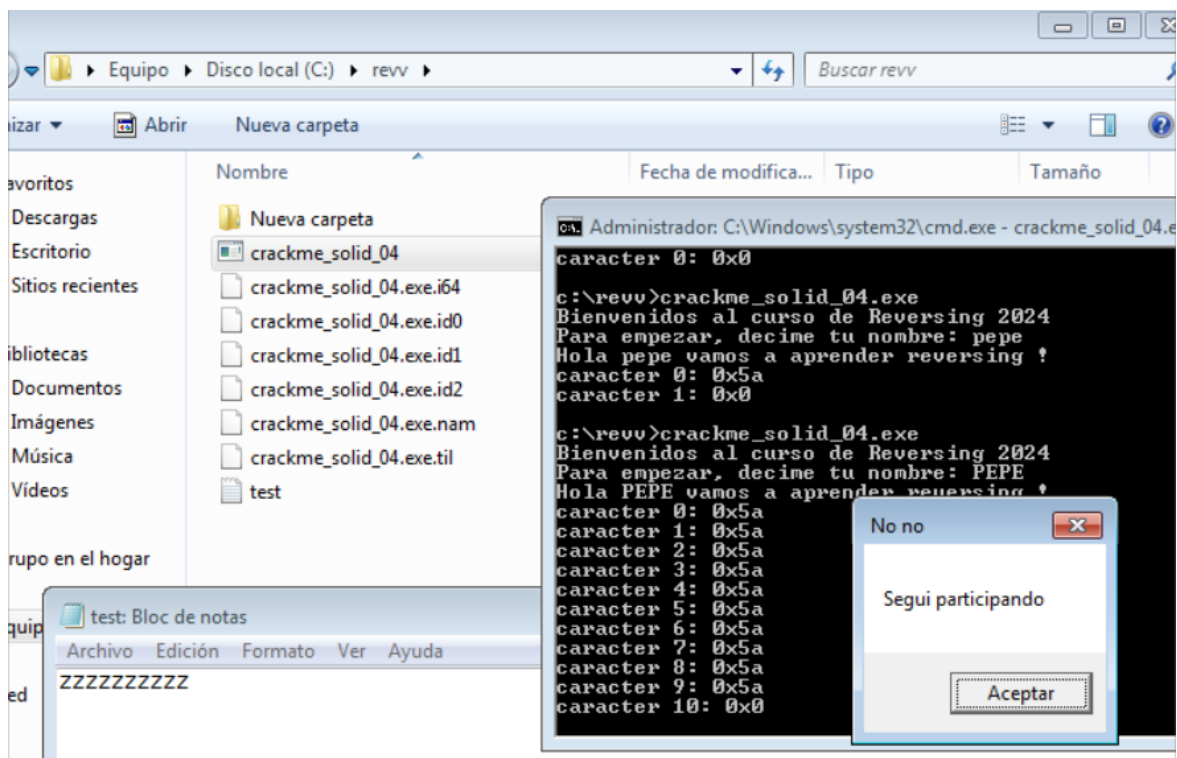
So, so far we have the following:

1. It asks you for a username—though so far I haven't found any operation being done with it.
2. It expects to find a test.txtfile with content (at first, I can guess it needs to have **Z** written in it).
3. And I think this is the most important part: it enters a loop, but it compares it with 0A (which is 10).

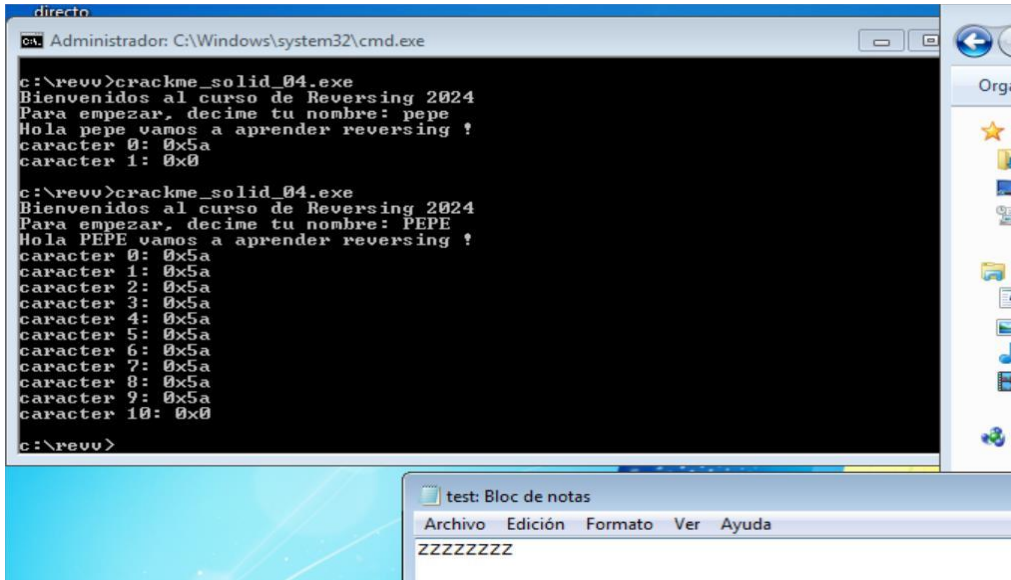
I don't think it's crazy to assume that it reads the file and loops through it 10 times, checking that each character is a **Z**. So why not try putting "ZZZZZZZZZZ"? Let's test it:



And I run it again.



Even though it has 10 'Z' characters, it still tells me "Keep trying"—but there's something weird here. It says character 0: 0x5a but it reaches character 9. In other words, it needs one more. Just like when you iterate through a list in Python or another language—it starts at index 0 and it's always the string length minus 1. So, instead of putting 10 Zs, I'm going to put 11 Zs.



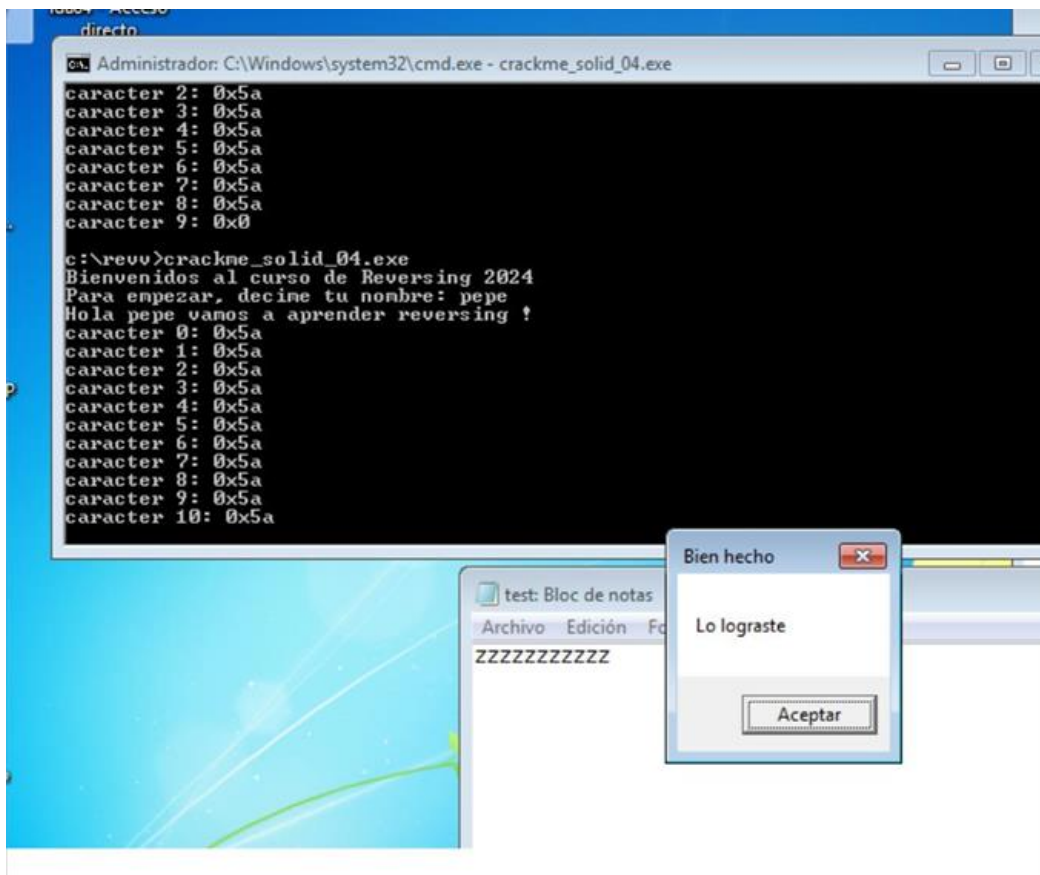
The screenshot shows a Windows XP desktop. In the foreground, a command prompt window titled "Administrador: C:\Windows\system32\cmd.exe" is open. It shows the execution of a program named "crackme_solid_04.exe". The program prompts the user for their name, and the user enters "pepe". It then displays the memory addresses for characters 0 through 10. Character 0 is 0x5a, and characters 1 through 9 are also 0x5a. Character 10 is 0x0. Below the command prompt, a Notepad window titled "test: Bloc de notas" is open, showing the text "ZZZZZZZZ".

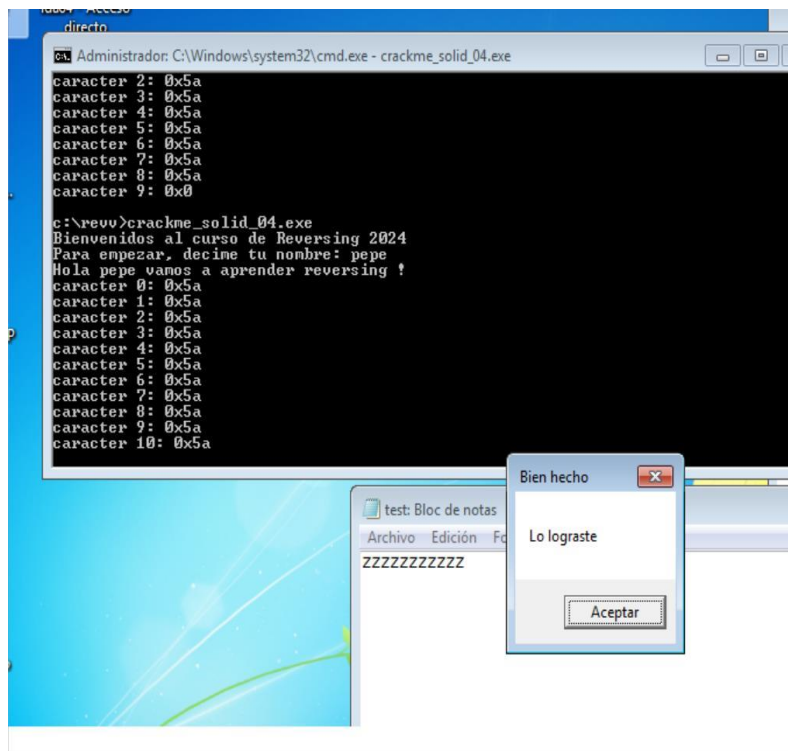
```
c:\revu>crackme_solid_04.exe
Bienvenidos al curso de Reversing 2024
Para empezar, decime tu nombre: pepe
Hola pepe vamos a aprender reversing !
caracter 0: 0x5a
caracter 1: 0x0

c:\revu>crackme_solid_04.exe
Bienvenidos al curso de Reversing 2024
Para empezar, decime tu nombre: PEPE
Hola PEPE vamos a aprender reversing !
caracter 0: 0x5a
caracter 1: 0x5a
caracter 2: 0x5a
caracter 3: 0x5a
caracter 4: 0x5a
caracter 5: 0x5a
caracter 6: 0x5a
caracter 7: 0x5a
caracter 8: 0x5a
caracter 9: 0x5a
caracter 10: 0x0

c:\revu>
```

And I'm going to run it:





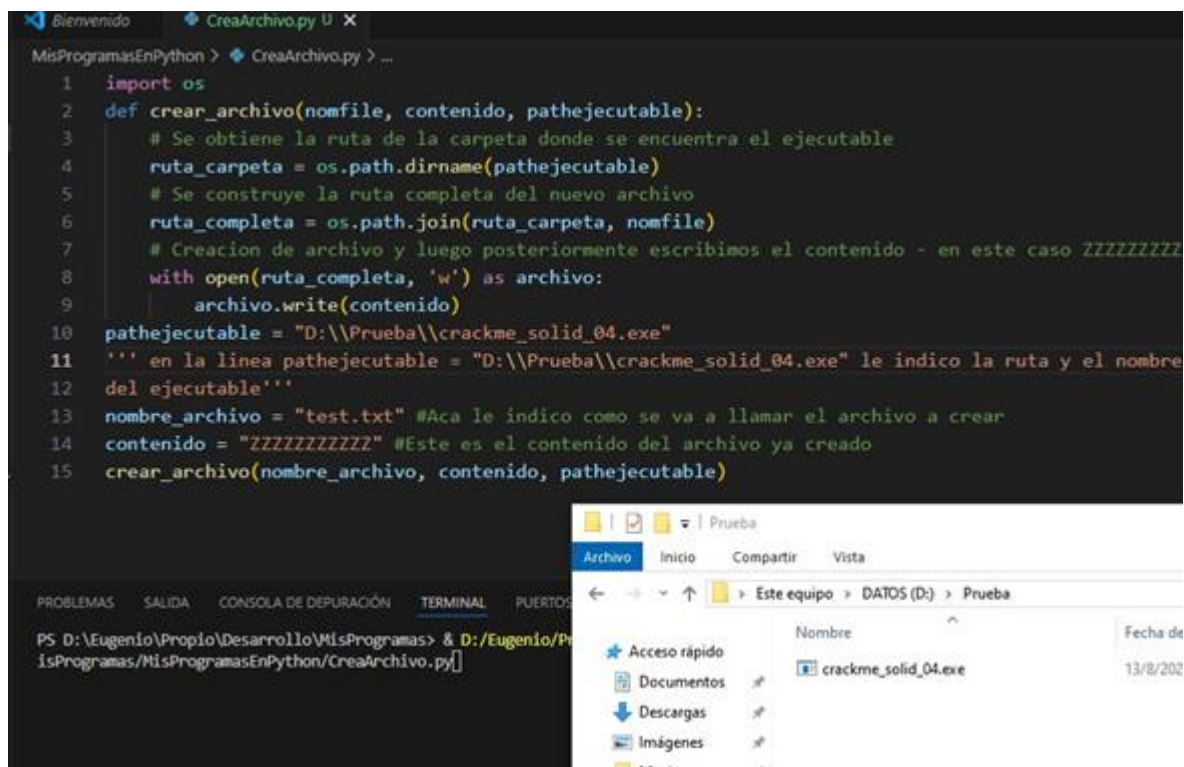
TADA! I did it! 🎉

But since I don't have much of an idea about programming either (well, I know a little bit, but not as much as I'd like, and I'm trying to learn Python), I'm going to generate a script. The idea is that if it finds an executable with a specific name in the specified directory, it will create a test.txt file with the content "ZZZZZZZZZZ".

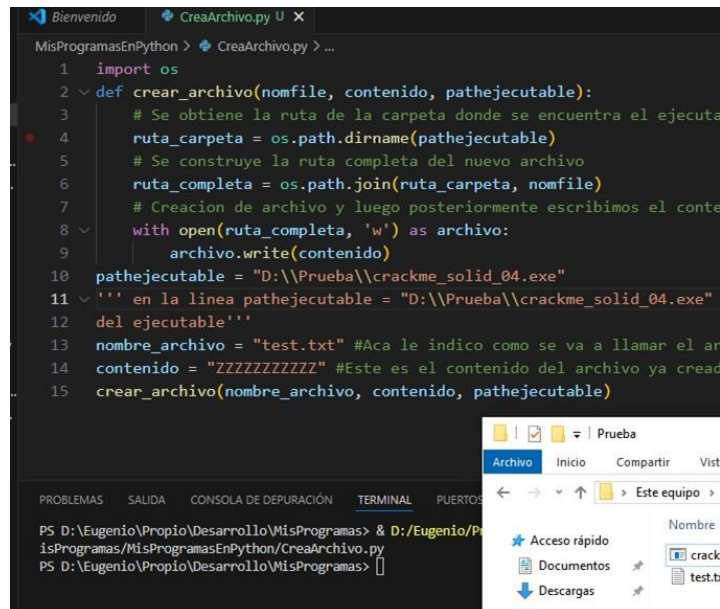
MisProgramasEnPython > CreaArchivo.py > ...

```
1  import os
2  def crear_archivo(nomfile, contenido, pathejecutable):
3      # Se obtiene la ruta de la carpeta donde se encuentra el ejecutable
4      ruta_carpeta = os.path.dirname(pathejecutable)
5      # Se construye la ruta completa del nuevo archivo
6      ruta_completa = os.path.join(ruta_carpeta, nomfile)
7      # Creacion de archivo y luego posteriormente escribimos el contenido
8      with open(ruta_completa, 'w') as archivo:
9          archivo.write(contenido)
10 pathejecutable = "D:\\Prueba\\crackme_solid_04.exe"
11 ''' en la linea pathejecutable = "D:\\Prueba\\crackme_solid_04.exe" le
12 del ejecutable'''
13 nombre_archivo = "test.txt" #Aca le indico como se va a llamar el arch
14 contenido = "ZZZZZZZZZZ" #Este es el contenido del archivo ya creado
15 crear_archivo(nombre_archivo, contenido, pathejecutable)
```


Testing it on my PC:



There, it created it with the corresponding content.

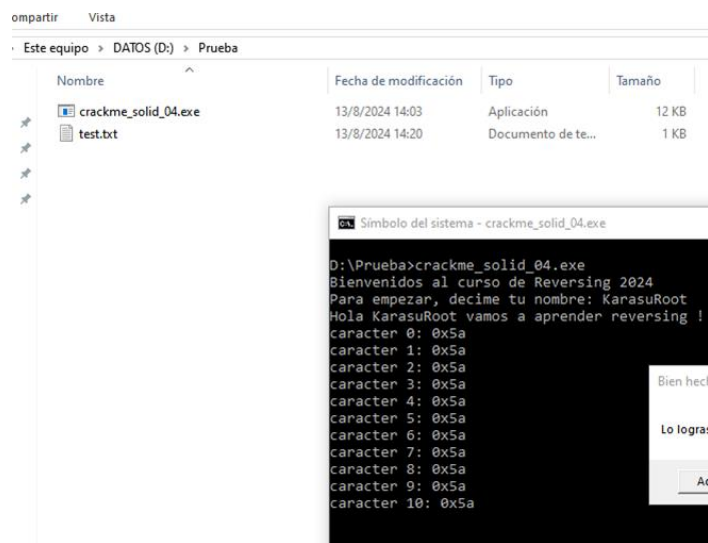


Nombre	Fecha de modificación	Tipo	Tamaño
crackme_solid_04.exe	13/8/2024 14:03	Aplicación	12 KB
test.txt	13/8/2024 14:20	Documento de te...	1 KB

test.txt: Bloc de notas

Archivo Edición Formato Ver Ayuda
ZZZZZZZZZZ

And it works!



I hope this is useful for those who are just starting out. Cheers!